

软件过程改进方案设计文档

学生：文就南

学号：2023141461219

改进对象：校园二手交易平台软件过程

1. 方案目标

本方案在原有瀑布-迭代混合式软件过程基础上，引入 AI 辅助节点，形成“人主导、AI 加速、评审闭环、指标度量”的智能化软件过程。改进目标包括：需求文档初稿效率提升 40%，测试用例设计效率提升 50%，代码样板与单元测试编写效率提升 30%，需求缺陷和测试遗漏尽量前移发现，同时保证安全、隐私、版权和可追溯性。

改进原则有四点。第一，AI 只作为辅助角色，所有关键决策由人类角色确认。第二，AI 输出必须进入配置管理，保留提示词、输入材料、输出版本和人工修改记录。第三，涉及用户隐私、交易安全、权限控制的内容必须经过人工复核。第四，过程指标既要记录效率，也要记录返工率、采纳率和缺陷逃逸率，避免只追求速度。

2. 改进前过程问题

原过程主要包括需求分析、概要设计、详细设计、编码实现、测试验证、部署运维和项目管理。其稳定性较好，但在课程项目中存在以下问题：

- 需求说明依赖人工整理，用户故事、验收标准和异常场景不完整。
- 设计文档与接口文档同步困难，评审时容易只讨论功能，不充分讨论安全和可维护性。
- 编码阶段重复劳动较多，DTO、校验逻辑、测试桩和注释占用大量时间。
- 测试用例覆盖不均，正常流程多，异常流程、边界值和权限场景少。
- 运维阶段缺少指标化监控，出现问题后依赖人工翻日志。

这些问题并非单个成员能力不足，而是过程本身缺少自动化检查和知识复用机制。AI 的引入应围绕这些薄弱点设计。

3. 改进后总体流程

flowchart TD

```
A[项目启动] --> B[需求访谈与资料收集]
B --> C[AI生成用户故事/验收标准/问题清单]
C --> D[需求负责人整理需求规格说明]
D --> E{需求评审}
E -- 修改 --> C
E -- 通过 --> F[AI辅助架构备选与接口草案]
F --> G[设计负责人完成人工设计决策]
G --> H{设计评审}
H -- 修改 --> F
H -- 通过 --> I[AI辅助编码/单元测试初稿]
I --> J[开发者修改、提交、代码审查]
J --> K[AI辅助测试用例与脚本生成]
```

```
K --> L[测试负责人评审并执行测试]
L --> M{质量门禁}
M -- 不通过 --> I
M -- 通过 --> N[部署与监控]
N --> O[AI辅助日志分析与改进建议]
O --> P[过程复盘与度量归档]
```

该流程保留原有评审节点，但把 AI 节点嵌入到“初稿生成、检查补全、辅助分析”位置。AI 不直接跨过评审，也不能直接发布生产变更。

4. 新增角色与职责

角色	职责	关键产出
AI 协同负责人	维护提示词模板、工具账号、输出归档规则，监督 AI 使用合规性	AI 使用记录、提示词库
AI 需求分析员	基于访谈纪要生成用户故事、验收标准、冲突清单	需求初稿、待确认问题
AI 测试分析师	生成测试用例、边界条件、自动化脚本初稿	测试用例矩阵、脚本草案
AI 代码审查助理	辅助检查可读性、异常处理、安全风险和重复代码	审查建议单
数据与安全责任人	检查输入 AI 的材料是否包含隐私、密钥、敏感数据	脱敏记录、风险确认

原有项目经理、需求负责人、架构师、开发者、测试负责人、运维负责人继续保留。新增 AI 角色可以由团队成员兼任，但职责必须写入过程文档。

5. 各环节改进方案

5.1 需求分析改进

输入包括访谈纪要、竞品功能、课程项目范围和用户角色。AI 生成用户故事、验收标准、异常场景和问题清单。需求负责人必须进行三类检查：真实性检查，即 AI 是否凭空增加未确认需求；可测性检查，即每条需求是否能转化为验收标准；一致性检查，即需求之间是否冲突。通过评审后形成《需求规格说明书》和《需求追踪矩阵》。

质量标准包括：每个核心功能至少包含 1 条主成功场景、2 条异常场景、1 条权限场景；每条用户故事必须对应验收标准；AI 生成内容的采纳率、修改率和废弃原因需记录。

5.2 系统设计改进

设计阶段使用 AI 生成架构备选方案、接口草案、数据表字段建议和风险清单。设计负责人负责最终决策。例如商品发布模块采用“前端表单校验 + 后端参数校验 + 图片格式校验 + 敏感词扫描 + 管理员抽检”的混合方案，而不是完全依赖 AI 审核。AI 输出的 UML 和接口草案只作为初稿，必须与项目技术栈、团队能力和时间约束匹配。

5.3 编码实现改进

编码阶段将 AI 用于样板代码、单元测试、注释、异常处理和局部重构。开发者提交代码前必须完成自查：是否存在硬编码密钥、越权访问、SQL 注入、空指针风险、异常吞掉、日志泄露隐私等问题。AI 代码建议必须经过人工理解后才能合并，不能“看起来能跑就提交”。所有 AI 生成的重要代码需在提交说明中标注辅助范围。

5.4 测试验证改进

测试阶段引入“AI 生成候选用例 + 人工评审 + 自动化脚本转化”的流程。AI 根据需求规格、接口契约和数据字典输出测试点，测试负责人按风险优先级排序。对于登录、商品发布、订单状态、举报审核等模块，应重点覆盖边界值、权限、异常输入、并发和数据一致性。测试通过标准不因 AI 参与而降低，缺陷管理仍按严重程度、复现步骤、期望结果、实际结果和修复状态记录。

5.5 运维监控改进

部署后采集接口耗时、错误率、登录失败率、图片上传失败率、订单异常率、数据库慢查询等指标。当异常超过阈值时，AI 读取脱敏后的日志片段、指标变化和最近发布记录，生成根因假设、影响范围和排查步骤。运维负责人确认后执行处理，并将结果写入故障复盘。

6. 工具链选型

环节	工具/框架	核心功能	集成方式	预期指标
需求	ChatGPT / Claude / 通义千问	需求整理、用户故事、验收标准	上传脱敏访谈纪要，按模板生成	初稿时间降低 40%
设计	ChatGPT + Mermaid	架构方案、流程图、接口草案	生成 Mermaid 与接口说明后人工修订	设计评审问题提前发现
编码	GitHub Copilot / Cursor / Codex	代码补全、重构、单元测试	接入 IDE 与仓库，限制敏感文件	样板代码时间降低 30%
测试	LLM + Postman / pytest	用例生成、接口脚本草案	从接口文档生成用例与脚本	用例设计时间降低 50%
运维	日志平台 + LLM	日志摘要、异常解释	输入脱敏日志和指标窗口	平均定位时间降低 30%

7. 风险评估与应对

风险	表现	等级	应对策略
数据安全	将用户手机号、聊天记录、密钥输入外部模型	高	输入前脱敏；禁止上传密钥；建立敏感数据清单
幻觉输出	AI 编造功能、接口或不存在的依赖	高	所有输出必须人工评审；引用来源；建立“待确认”标记
代码安全	生成越权、注入、弱加密代码	高	安全检查清单、代码审查、静态扫描
技术依赖	团队过度依赖 AI，理解能力下降	中	要求解释关键代码；评审时抽查设计理由
版权合规	生成代码来源不明	中	避免整段复制未知代码；保留生成记录；使用开源许可检查
质量漂移	AI 输出风格不一致，文档版本混乱	中	提示词模板化；输出纳入版本管理

8. 度量指标

改进后的过程指标分为效率、质量、AI 贡献和风险四类。效率指标包括需求初稿耗时、用例设计耗时、单元测试编写耗时、缺陷定位耗时。质量指标包括需求评审缺陷数、测试用例覆盖率、缺陷逃逸率、代码审查问题数。AI 贡献指标包括 AI 输出采纳率、人工修改率、AI 生成用例有效率。风险指标包括敏感信息拦截次数、AI 幻觉纠正次数、安全审查问题数。

9. 实施计划

第一周完成 AI 场景匹配和提示词模板；第二周在需求和测试阶段试运行，形成第一版过程规范；第三周完成商品发布模块的 AI 测试验证；第四周根据度量结果修订完整过程文档。最终形成四类资产：场景匹配报告、改进方案、验证报告和完整过程定义文档。

10. 方案总结

本方案的重点不是追求“全自动开发”，而是把 AI 放入可管理的软件过程。AI 负责提高初稿生成、检查补全和分析效率，人类负责业务判断、质量确认和风险承担。通过角色、流程、工具、质量标准和度量指标的同步调整，校园二手交易平台的软件过程能够从传统阶段式流程升级为人机协同流程。

11. RACI 职责矩阵

为了保证 AI 节点不会造成责任模糊，本方案引入 RACI 矩阵。其中 R 表示负责执行，A 表示最终负责，C 表示参与咨询，I 表示需要知会。

活动	项目经理	需求负责人	设计负责人	开发负责人	测试负责人	AI协同负责人	数据安全负责人
---	---	---	---	---	---	---	---
需求资料收集	A	R	C	I	C	C	C
AI 需求初稿生成	I	A	I	I	C	R	C
需求评审	A	R	C	C	C	C	C
AI 架构备选生成	I	C	A/R	C	C	C	C
代码生成与单测辅助	I	I	C	A/R	C	C	C
AI 测试用例生成	I	C	C	C	A/R	R	C
敏感数据脱敏检查	I	C	C	C	C	C	A/R
发布评审	A	I	C	R	R	I	C

该矩阵明确了一个关键原则：AI 协同负责人负责工具和过程，不对业务正确性单独负责。业务正确性仍由需求、设计、开发、测试等原有负责人承担。

12. 质量门禁细化

改进后的质量门禁分为四类。需求门禁要求所有 P0/P1 需求必须具备验收标准和追踪编号，AI 生成的待确认问题必须关闭。设计门禁要求关键接口、数据库约束、权限策略和异常处理有明确说明，AI 提出的风险必须被接受、规避或说明暂不处理。代码门禁要求所有代码通过本地测试、静态检查和同伴审查，AI 生成代码必须能被开发者解释。测试门禁要求核心用例执行通过，严重缺陷关闭，回归测试完成。

门禁	检查项	不通过示例	处理方式
---	---	---	---
需求门禁	验收标准完整	“支持发布商品”没有边界规则	退回需求补充
设计门禁	权限与状态清晰	未说明未登录发布接口行为	退回设计修订
代码门禁	安全与可维护性	AI 生成 SQL 拼接代码	拒绝合并
测试门禁	覆盖高风险路径	只测正常发布，不测图片异常	补充用例
发布门禁	可回滚可监控	没有部署说明和回滚方案	暂停发布

13. AI 输出物质量标准

AI 输出物不能只看“像不像”，必须有可检查标准。需求类输出要求需求编号清晰、业务术语一致、不得新增未经确认的功能；设计类输出要求与项目技术栈一致，不得引入团队无法实现的组件；代码类输出要求可运行、可测试、可解释；测试类输出要求步骤明确、数据具体、预期结果可判定；运维类输出要求只作为根因假设，不得直接当作结论。

此外，每份 AI 输出都应具备三种状态：采纳、修改后采纳、废弃。废弃时需要记录原因，例如“业务不适用”“与现有设计冲突”“存在安全风险”“描述重复”“无法复现”。这些记录可以帮助团队评估 AI 的真实贡献。

14. 过程落地计划

落地计划分四步推进。第一步是准备阶段，建立提示词模板、脱敏规则、AI 输出记录表和工具使用规范。第二步是试运行阶段，选择商品发布模块作为样例，把 AI 用于需求补全和测试用例生成。第三步是扩展阶段，把成熟做法推广到登录、订单、举报审核等模块。第四步是固化阶段，将有效实践写入完整软件过程定义文档，并在复盘会中持续优化。

周次	主要任务	交付物	验收标准
---	---	---	---
第1周	调研与场景选择	场景匹配报告	至少识别 3 个优先场景
第2周	设计改进流程	改进方案与风险清单	流程图、角色、工具链完整
第3周	原型验证	验证报告	有对比数据和结论
第4周	文档整合	完整过程定义文档	覆盖全生命周期

15. 经济性与可行性分析

从成本角度看，本方案优先选择低成本工具和轻量流程。通用大模型和 IDE 插件的学习成本低，能快速嵌入课程项目；Mermaid、Markdown、Git 等工具也符合课程交付要求。真正的成本不在工具采购，而在规范 AI 使用：需要额外维护提示词、审查输出、记录采纳情况和处理风险。但这些成本能够换来更完整的需求、更高的测试覆盖和更清晰的复盘材料。

从可行性角度看，本方案避免引入大型平台和复杂自动化流水线，适合学生团队执行。即使没有企业级 AI 测试平台，也可以通过“LLM 生成候选用例 + 人工评审 + Postman/pytest 转化”的方式完成验证。因此，本方案具有较好的课程可交付性和真实项目迁移价值。

16. 验收标准

本方案最终验收可以从四个方面进行。第一，文档完整性：是否包含场景分析、改进流程、角色职责、工具链、风险评估和度量指标。第二，过程可执行性：团队成员是否能根据流程完成一次模块开发。第三，质量可证明性：是否有 AI 辅助前后的效率和质量对比数据。第四，风险可控性：是否有脱敏、评审、审查和追溯机制。若四项均满足，则说明 AI 改进不是停留在概念层面，而是具备落地价值。

参考文献与资料

1. Cheng H., Husen J. H., Lu Y. 等. Generative AI for Requirements Engineering: A Systematic Literature Review, arXiv, 2024. <https://arxiv.org/abs/2409.06741>

2. Huang Y., Chen Y., Chen X. 等. Generative Software Engineering, arXiv, 2024.
<https://arxiv.org/abs/2403.02583>
3. Pandey R., Singh P., Wei R., Shankar S. Transforming Software Development: Evaluating the Efficiency and Challenges of GitHub Copilot in Real-World Projects, arXiv, 2024.
<https://arxiv.org/abs/2406.17910>
4. Song F., Agarwal A., Wen W. The Impact of Generative AI on Collaborative Open-Source Software Development: Evidence from GitHub Copilot, arXiv, 2024.
<https://arxiv.org/abs/2410.02091>
5. Li H., Zhang H., Hassan A. E. The Rise of AI Teammates in Software Engineering (SE) 3.0, arXiv, 2025. <https://arxiv.org/abs/2507.15003>
6. OpenAI. Introducing Codex, 2025. <https://openai.com/index/introducing-codex/>
7. GitHub Docs. GitHub Copilot documentation, 2025-2026. <https://docs.github.com/en/copilot>
8. Microsoft. Playwright MCP documentation, 2025-2026. <https://playwright.dev/>